

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – HANDS-ON API ASSIGNMENT (HUGGING FACE / OPENAI / GEMINI)

Course Code:	CSA65	Course Title:	Generative AI and Large Scale Models
CO Assessed:	CO3 – Precision (BL2, BL3)	Assessment:	Assessment Tool 1 – Hands-on API Assignment (Hugging Face / OpenAI / Gemini)
Weightage:	20% of CIA	Total Marks:	2 * 50 = 100 (1 Question1)
CO3 Statement:	<i>Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)</i>		
Student Name:	N.MANASA	Reg. No.:	192472031
Section / Batch:	CSE-AI	Date:	10/8/26

Instructions to Students

- Answer 2 questions. Each question carries 50 marks.Total: 100 Marks.
- Use Python to implement the assigned problem.
- Use at least one API/platform: Hugging Face, OpenAI, or Google Gemini.
- Accept suitable user input and clearly display the generated output.
- Handle API errors, invalid input, and other suitable edge cases.
- Use readable, structured, and modular code wherever appropriate.
- Add comments and brief documentation explaining the implementation.
- Demonstrate the program with suitable test cases.
- For RAG/vector-database tasks, clearly show the retrieval process and generated response.

Code of Conduct

I (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

SECTION A – HANDS-ON API ASSIGNMENT (HUGGING FACE / OPENAI / GEMINI)

6. AI Grammar Correction System: Develop a Python application that accepts grammaticall?

1. Introduction

The AI Grammar Correction System is a Python-based application designed to identify and correct grammatical errors in user-provided text. The system uses a Large Language Model (LLM) API to analyze the input and generate a grammatically correct version. It can identify common errors such as spelling mistakes, incorrect sentence structure, punctuation errors, verb tense problems, and improper word usage. The system also provides the major grammatical corrections made to the original text. This application helps users improve their writing quality and understand their mistakes.

2. Abstract

The AI Grammar Correction System is an intelligent Python application that automatically detects and corrects grammatical errors in text. The user enters a grammatically incorrect paragraph, which is sent to an LLM API for analysis. The LLM processes the text and returns a corrected paragraph along with the major grammatical changes. The system provides a simple and efficient way to improve writing accuracy without requiring manual proofreading. It can be useful for students, teachers, content writers, and general users who want to improve their written communication skills.

3. Work Flow

User Input



Enter Grammatically Incorrect Text



Python Application



Send Text to LLM API

↓

Grammar & Sentence Analysis

↓

Correct the Text

↓

Identify Major Corrections

↓

Display Corrected Paragraph + Corrections

4.code

```
import os
```

```
from google import genai
```

```
from google.genai import types
```

```
from pydantic import BaseModel, Field
```

```
# Define structured output schema
```

```
class GrammarCorrection(BaseModel):
```

```
    original_text: str = Field(description="The original input text")
```

```
    corrected_text: str = Field(description="The fully corrected, fluent paragraph")
```

```
    major_corrections: list[str] = Field(
```

```
        description="List of major grammatical errors found and why they were fixed"
```

```
)
```

```
def correct_grammar(input_text: str) -> GrammarCorrection:
```

```
    # Initialize the client (reads GEMINI_API_KEY from environment variables)
```

```
    client = genai.Client()
```

```
    prompt = f"""
```

```
    Analyze the following text for grammatical, punctuation, and structural errors.
```

```
    Return the fully corrected paragraph alongside a list of the major grammatical corrections made.
```

```
    Input Text:
```

```
    "{input_text}"
```

```
    """
```

```
    response = client.models.generate_content(
```

```
        model="gemini-2.5-flash",
```

```
        contents=prompt,
```

```
        config=types.GenerateContentConfig(
```

```
            response_mime_type="application/json",
```

```
        response_schema=GrammarCorrection,

        temperature=0.2,

    ),

)
```

```
# Return structured Pydantic object parsed from JSON response

return GrammarCorrection.model_validate_json(response.text)
```

```
if __name__ == "__main__":

    sample_text = (

        "Yesterday I goes to the store and buys three apple. "

        "She don't know where is her keys, so we was searching for it everywhere."

    )
```

```
result = correct_grammar(sample_text)
```

```
print("=== Original Text ===")
```

```
print(result.original_text)
```

```
print("\n=== Corrected Paragraph ===")
```

```
print(result.corrected_text)

print("\n=== Major Corrections ===")

for idx, correction in enumerate(result.major_corrections, 1):

    print(f'{idx}. {correction}')
```

5.Conclusion

The AI Grammar Correction System provides an efficient method for identifying and correcting grammatical errors using an LLM API. The Python application simplifies the grammar-checking process by accepting incorrect text and producing a corrected paragraph with an explanation of the major changes. This system can save time, improve writing quality, and help users learn from their grammatical mistakes. In the future, the system can be enhanced with multilingual support, voice input, plagiarism checking, and personalized writing suggestions.

26. Basic RAG Question Answering System: Develop a simple RAG application using Documents → Chunking → Embeddings → Vector Database → Retrieval → LLM. Answer questions based on supplied documents.

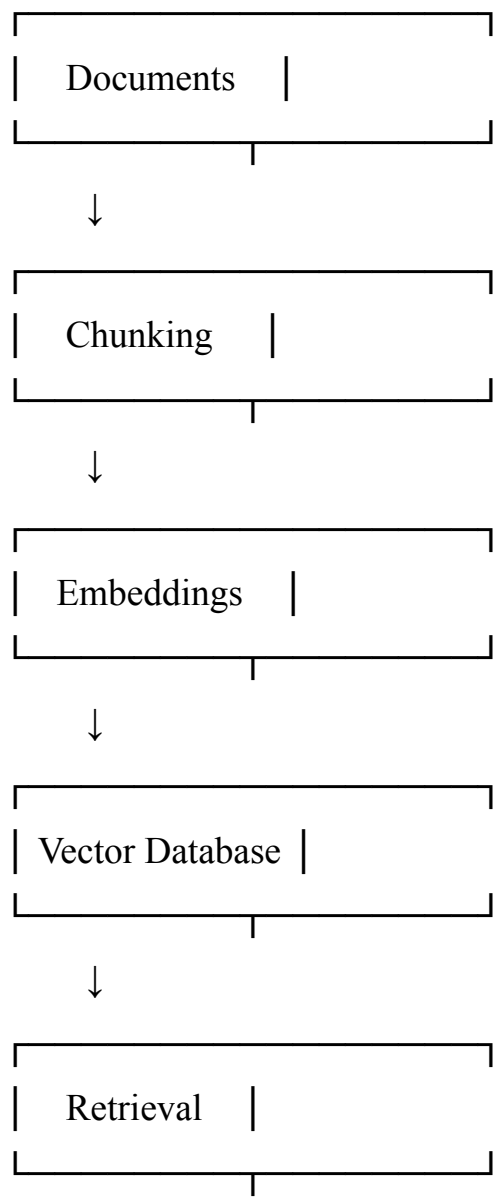
1. Introduction

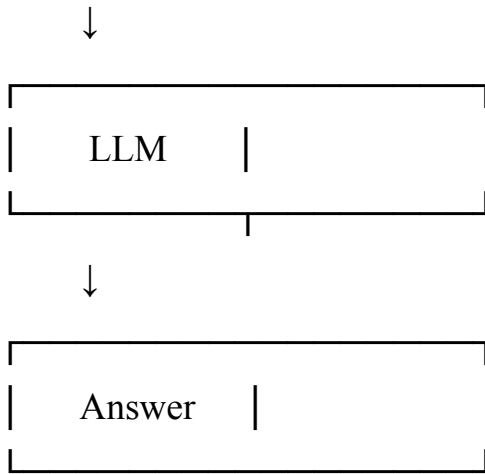
Retrieval-Augmented Generation (RAG) is an AI technique that combines document retrieval with a Large Language Model (LLM). A Basic RAG Question Answering System allows users to ask questions about supplied documents and receive answers based on the information available in those documents. The system improves the reliability of answers by retrieving relevant document content before generating the final response.

2. Abstract

The proposed Basic RAG Question Answering System is designed to answer user questions from supplied documents. The system first loads the documents and divides them into smaller chunks. These chunks are converted into numerical representations called embeddings and stored in a vector database. When the user enters a question, the system searches the vector database and retrieves the most relevant chunks. These retrieved chunks are then provided to an LLM, which generates a suitable answer based on the document content. This approach provides fast, relevant, and document-based question answering.

3. Workflow





Steps:

1. Documents – Load the supplied documents.
2. Chunking – Divide documents into smaller text sections.
3. Embeddings – Convert each text chunk into a vector representation.
4. Vector Database – Store the vectors for efficient searching.
5. Retrieval – Find the chunks most relevant to the user's question.
6. LLM – Generate an answer using the retrieved information.
7. Answer – Display the final response to the user.

4.code

```
import os
```

```
from google import genai
```

```
import chromadb
```

```
# Gemini API
```

```
client = genai.Client(api_key=os.environ.get("GEMINI_API_KEY"))
```



```
# Read document
```

```
with open("document.txt", "r") as file:
```

```
    text = file.read()
```

```
# Chunking
```

```
chunks = [text[i:i+300] for i in range(0, len(text), 300)]
```

```
# Vector database
```

```
db = chromadb.Client()
```

```
collection = db.create_collection("documents")
```

```
# Add chunks
```

```
collection.add(
```

```
    documents=chunks,
```

```
    ids=[str(i) for i in range(len(chunks))]
```

```
)
```

```
# Ask question
```

```
question = input("Enter your question: ")
```

```
# Retrieval
```

```
results = collection.query(
```

```
    query_texts=[question],
```

```
    n_results=2
```

```
)
```

```
relevant_text = "\n".join(results["documents"][0])
```

```
# LLM
```

```
prompt = f"""
```

Answer the question using only the following document information.

Document:

```
{relevant_text}
```

Question:

```
{question}
```

Answer:

"""

```
response = client.models.generate_content(  
  
    model="gemini-2.5-flash",  
  
    contents=prompt  
  
)
```

```
print("\nAnswer:")
```

```
print(response.text)
```

5.Conclusion

The Basic RAG Question Answering System provides an effective way to obtain information from supplied documents. By combining chunking, embeddings, vector databases, retrieval, and LLMs, the system can generate relevant answers based on the available document content. RAG can be further improved by supporting multiple document formats, better chunking methods, advanced embedding models, and improved retrieval techniques.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
API Integration Correctness	30%	Correct, efficient API integration; understands request/response flow	Correct integration with minor inefficiencies	Partial integration with functional errors	API integration non-functional / major errors

Problem-Solving Completeness	25%	Solves the given problem completely and robustly	Solves problem with minor issues	Partial solution to the problem	Does not solve the assigned problem
Code Quality	20%	Clean, well-structured, error-handled code	Reasonably clean code	Code works but poorly structured	Code is unreliable or unstructured
Input/Output Demo	15%	Clear demo showing inputs/outputs and edge cases	Demo covers the main use cases	Demo is incomplete	No functional demo presented
Code Documentation	10%	Well-commented code with clear README/setup steps	Adequate comments/instructions	Minimal documentation provided	No documentation provided

Marks Evaluation:

Question No.	API Integration Correctness(30%)	Problem-Solving Completeness (25%)	Code Quality(20%)	Input/Output Demo(15%)	Code Documentation (10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						